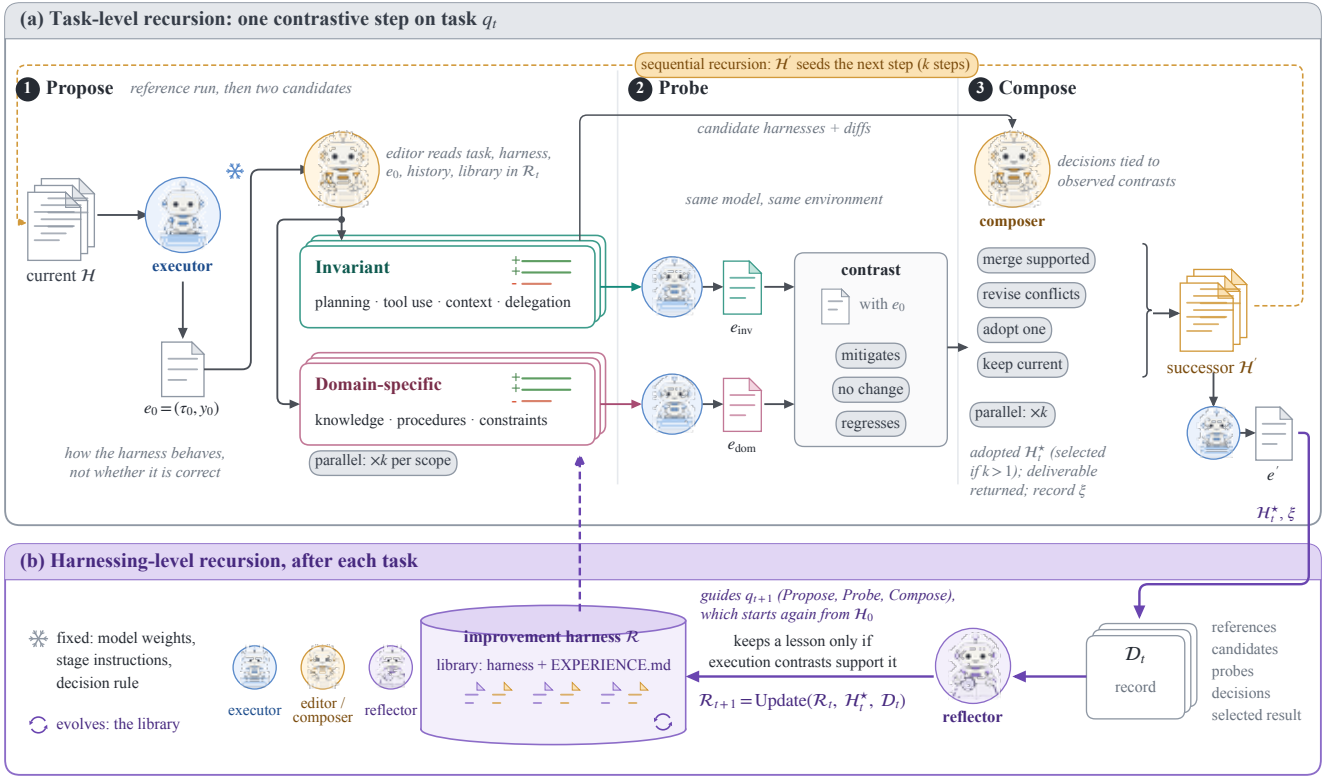

(a) Task-level recursion: one contrastive step on task q_t
sequential recursion: $\mathcal{H}'$ seeds the next step (k steps)
1 Propose
reference run, then two candidates
2 Probe
3 Compose
current $\mathcal{H}$
executor
$e_0 = (\tau_0, y_0)$
how the harness behaves, not whether it is correct
editor reads task, harness, e_0, history, library in $\mathcal{R}_t$
Invariant
planning · tool use · context · delegation
Domain-specific
knowledge · procedures · constraints
parallel: ×k per scope
candidate harnesses + diffs
same model, same environment
e_{inv}
e_{dom}
contrast
with e_0
mitigates
no change
regresses
composer
decisions tied to observed contrasts
merge supported
revise conflicts
adopt one
keep current
parallel: ×k
adopted $\mathcal{H}_t^\star$ (selected if $k>1$); deliverable returned; record ξ
successor $\mathcal{H}'$
e'
$\mathcal{H}_t^\star, \xi$
(b) Harnessing-level recursion, after each task
fixed: model weights, stage instructions, decision rule
evolves: the library
executor
editor / composer
reflector
improvement harness $\mathcal{R}$
library: harness + EXPERIENCE.md
guides q_{t+1} (Propose, Probe, Compose), which starts again from $\mathcal{H}_0$
keeps a lesson only if execution contrasts support it
$\mathcal{R}_{t+1} = \text{Update}(\mathcal{R}_t,\ \mathcal{H}_t^\star,\ D_t)$
reflector
D_t
record
references
candidates
probes
decisions
selected result